Name: Sk Arshad Basha

RegNo: 192312336

1. You are working as a data analyst in a hospital where patient data is continuously and stored in CSV files. However, the dataset contains missing values, inconsistent units, and duplicate entries. The system must process this data in near real-time for reporting. Explain how you would design a solution using NumPy and Pandas to clean, filter, and standardize the data efficiently. Discuss indexing, aggregation, and saving the processed data, along with strategies to handle unexpected data anomalies.

Introduction

Hospital patient data is continuously collected from different sources and stored in CSV files. Such data may contain missing values, inconsistent units, duplicate entries, invalid records, and unexpected anomalies. Therefore, an efficient data-processing system is required to clean, standardize, validate, and organize the information before it is used for reporting. NumPy and Pandas provide efficient tools for numerical calculations and DataFrame-based data processing.

Objective

- To clean continuously collected patient data.
- To handle missing values and duplicate records.
- To standardize inconsistent units and formats.
- To identify invalid and abnormal records.
- To generate accurate reports efficiently.

Data Loading and Cleaning

Pandas can be used to read CSV files and convert them into DataFrames. Missing numerical values can be handled using suitable statistical measures such as mean

or median, while missing categorical values can be replaced with the mode or an appropriate category. Duplicate records should be identified using important fields such as patient ID, date, and time.

Indexing and Filtering

Indexing can be performed using Patient ID, date, or department to provide efficient access to records. Filtering can be used to identify invalid medical values. For example, values outside a defined valid range can be flagged for further examination. Boolean indexing allows multiple conditions to be applied efficiently.

Standardization

Different sources may use different units for the same measurement. These values should be converted into a common unit before analysis. Categorical values should also be standardized by removing unnecessary spaces and using consistent naming conventions.

Aggregation

Pandas aggregation functions such as count, mean, sum, minimum, and maximum can be used to generate reports. Data can be grouped according to department, gender, age group, or other relevant fields.

Saving Processed Data

After cleaning and validation, the processed DataFrame can be saved as a new CSV file. This preserves the original raw data while providing a reliable dataset for reporting.

Anomaly Handling

Unexpected anomalies should be detected using range checks and statistical analysis. Important unusual records should be flagged rather than automatically deleted. This helps preserve potentially important patient information.

Conclusion

A NumPy and Pandas-based solution can efficiently clean, filter, standardize, aggregate, and save hospital data. Proper validation and anomaly handling improve data quality and support reliable near-real-time reporting.

2. A financial company provides you with millions of transaction records that must be to detect unusual spending patterns. Due to system limitations, memory and computation time are critical constraints. Describe how you would use NumPy arrays and Pandas DataFrames to perform filtering, grouping, and statistical analysis efficiently. Explain how you would optimize indexing and aggregation operations, and how you would debug performance bottlenecks or incorrect outputs.

Introduction

Financial companies generate millions of transaction records that must be analyzed to understand customer spending and identify unusual transactions. Since the dataset is very large, memory usage and computation time are important constraints. NumPy and Pandas provide efficient array and DataFrame operations for processing such large datasets.

Objective

- To analyze millions of transaction records.
- To identify unusual spending patterns.
- To minimize memory consumption.
- To perform efficient filtering and aggregation.

- To identify and debug performance problems.

Data Processing

Pandas DataFrames can store transaction records containing customer IDs, transaction amounts, dates, and categories. Only the required columns should be loaded when possible. For extremely large files, data can be processed in smaller chunks rather than loading the complete dataset into memory.

Filtering

Filtering can identify transactions satisfying particular conditions. For example, unusually high transaction amounts can be selected for further analysis. Multiple Boolean conditions can be combined to identify suspicious spending patterns.

Indexing

Customer ID, account ID, or transaction date can be used as an index depending on the analysis requirements. Proper indexing provides efficient access to frequently searched records. However, indexes should be used carefully because they also consume memory.

Grouping and Aggregation

Transactions can be grouped by customer, account, category, or location. Aggregation functions can calculate total spending, average spending, transaction count, minimum, maximum, variance, and standard deviation.

Statistical Analysis

NumPy can efficiently calculate statistical measures from numerical transaction arrays. Mean and standard deviation can help identify transactions that significantly differ from normal customer behavior.

Memory Optimization

Memory can be reduced by:

- Loading only required columns.
- Using suitable numerical data types.
- Processing large files in chunks.
- Avoiding unnecessary DataFrame copies.
- Filtering before expensive operations.
- Using vectorized NumPy and Pandas operations.

Debugging

Performance can be investigated by checking memory usage and execution time. Incorrect results can be identified by checking data types, missing values, duplicate transactions, record counts, and intermediate aggregation results.

Conclusion

Efficient use of NumPy and Pandas enables millions of financial transactions to be processed within memory and time constraints. Proper indexing, filtering, grouping, aggregation, optimization, and debugging provide accurate and efficient financial analysis.

3. You are given multiple datasets from different departments (sales, marketing, and customer support), each stored in different file formats. Your task is to combine these datasets into a unified DataFrame while ensuring no loss of critical information and maintaining consistent indexing. Explain your approach using Pandas for file I/O, combining, grouping, and sorting. How would you validate

Introduction

Organizations often maintain separate datasets for sales, marketing, and customer support. These datasets may be stored in different file formats and may contain different column structures, data types, missing values, and duplicate keys.

Combining them into a unified DataFrame allows complete information to be analyzed together.

Objective

- To combine datasets from different departments.
- To preserve all important records.
- To maintain consistent indexing.
- To identify duplicate and missing keys.
- To validate the final merged dataset.

File Input and Standardization

Pandas provides functions for reading CSV, Excel, JSON, and other supported file formats. After loading the datasets, column names and data types should be examined. Common fields should be standardized before combining the DataFrames.

Common Key

A common key such as **Customer ID** can be used to connect the datasets. The key should have the same data type and format in every DataFrame. Spaces, capitalization differences, and inconsistent representations should be corrected.

Combining DataFrames

Pandas provides **merge, join, and concat** operations. A suitable merge method should be selected according to the requirement. An outer merge can preserve records from all departments, including records that do not have matching entries in another dataset.

Duplicate and Missing Keys

Duplicate keys should be detected before merging. However, duplicates should not automatically be deleted because multiple sales transactions for one customer may be valid. Missing and unmatched keys should be identified and investigated.

Grouping and Sorting

After merging, records can be grouped by customer, department, product, or other categories. Aggregation functions can calculate total sales, number of support requests, or marketing interactions. Sorting can organize the final data for reporting.

Validation

The merged dataset should be checked for:

- Missing records.
- Duplicate records.
- Unmatched keys.
- Incorrect data types.
- Unexpected changes in record count.
- Conflicting values.

Conclusion

Pandas provides an effective solution for combining heterogeneous departmental datasets. Standardization, correct merging, indexing, grouping, sorting, and validation ensure that critical information is preserved and the final DataFrame remains consistent.

4. A company requires an automated system that reads daily data files, processes them, and generates summary reports along with visualizations. The pipeline must filter relevant data, compute statistics, and produce sorted outputs for

dashboards. how you would design this workflow using NumPy and Pandas, including file handling, filtering, aggregation, and plotting. Also explain how you would handle errors such as corrupted files, inconsistent formats, or missing columns.

Introduction

Organizations frequently receive new data files every day and require automated reports and visualizations for decision-making. A complete data-analysis pipeline should automatically load the files, validate and clean the data, filter relevant records, calculate statistics, generate summaries, and produce visualizations.

Objective

- To automate daily data processing.
- To handle multiple data files efficiently.
- To filter relevant information.
- To calculate statistical measures.
- To generate reports and visualizations.

File Handling

The pipeline first identifies the daily data files and reads them using Pandas. Each file should be checked for corruption, empty content, incorrect formats, and missing required columns. Valid DataFrames can then be combined into a single dataset.

Data Cleaning

Missing values, duplicate records, inconsistent formats, and incorrect data types should be handled before analysis. Numerical columns can be converted into appropriate numerical types, while categorical columns can be standardized.

Filtering

Filtering is used to select only relevant records based on user-defined conditions. Boolean indexing can apply multiple conditions efficiently. Filtering before aggregation can reduce the amount of data that must be processed.

Aggregation

The filtered data can be grouped according to categories such as product, region, department, or date. Functions such as **sum, mean, count, minimum, and maximum** can be used to calculate summary information.

Sorting

The aggregated results can be sorted according to important measures such as total sales or number of transactions. This makes the results easier to understand and suitable for dashboards.

Visualization

The summarized information can be displayed using suitable plots:

- **Bar chart** – category comparison.
- **Line chart** – trends over time.
- **Histogram** – data distribution.
- **Pie chart** – proportions when appropriate.

Error Handling

The pipeline should handle corrupted files, missing columns, invalid values, empty files, and inconsistent formats. Errors should be reported and logged without stopping the processing of all valid files.

Conclusion

An automated NumPy and Pandas workflow provides a systematic approach for converting daily raw files into clean datasets, statistical summaries, reports, and meaningful visualizations. It improves efficiency, consistency, and reliability.

5. You are working on an e-commerce platform where customer behavior data is collected in real time. The dataset contains clickstream data with missing values and inconsistent formats. Explain how you would preprocess and structure this data using Pandas. Discuss filtering, feature extraction, indexing strategies, and how you would prepare the data for downstream machine learning tasks.

Introduction

E-commerce platforms continuously collect clickstream data about customer activities such as page views, product clicks, searches, sessions, and purchases. This data provides valuable information about customer behavior but may contain missing values, duplicate events, inconsistent formats, and invalid records. Proper preprocessing is therefore required before the data is used for analysis and machine learning.

Objective

- To clean clickstream data.
- To handle missing and inconsistent values.
- To remove duplicate and invalid events.
- To extract useful customer-behavior features.
- To prepare structured data for machine learning.

Data Preprocessing

Pandas DataFrames can be used to organize clickstream records and perform cleaning operations. Missing values should be identified and handled using

appropriate techniques. Numerical values should be converted into suitable data types, while categorical values should be standardized.

Filtering

Filtering can remove invalid records such as negative session duration or invalid timestamps. Duplicate events should also be identified using event IDs or other suitable identifiers. Boolean indexing can efficiently select valid records.

Feature Extraction

Important features can be extracted from clickstream data. Examples include:

- Number of page views.
- Number of product clicks.
- Session duration.
- Number of purchases.
- Activity hour.
- Day of the week.
- Number of interactions per user.

These features provide useful information about customer behavior.

Indexing

User ID can be used for customer-level analysis, while timestamp-based indexing can support time-based analysis. An appropriate index improves access to frequently analyzed records.

Grouping and Aggregation

Clickstream records can be grouped by user ID or session ID. Aggregation can calculate total events, total browsing time, purchase counts, and other behavioral

measures. These summarized features can be used as inputs to machine-learning models.

Machine Learning Preparation

Before machine learning, the dataset should have consistent formats, valid numerical values, handled missing values, and standardized categorical features. Categorical data may need to be encoded, while irrelevant columns should be removed. The final dataset should be structured into suitable features and target variables.

Validation

The final dataset should be checked for:

- Missing values.
- Duplicate events.
- Invalid values.
- Incorrect data types.
- Inconsistent categories.
- Incorrect timestamps.

Conclusion

Pandas and NumPy provide powerful tools for preprocessing e-commerce clickstream data. Filtering, indexing, feature extraction, grouping, aggregation, and standardization transform raw clickstream records into reliable features suitable for downstream machine-learning applications.